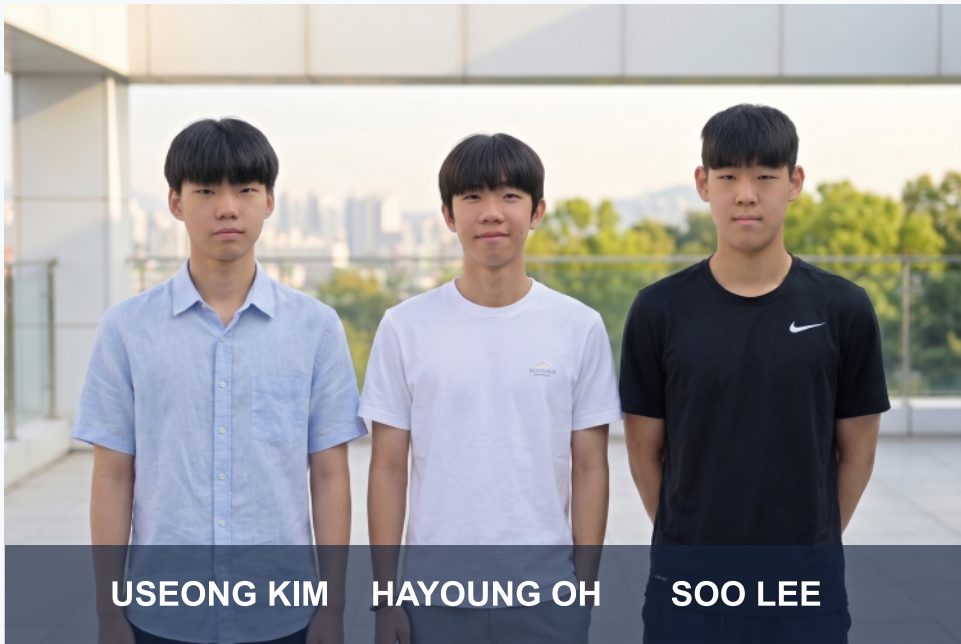


TEAM TOP CLASS

Autonomous maze rescue robot | League: Rescue Maze | South Korea

M5Stack Core S3 Lite • BaseX + 4x LEGO EV3 motors • 4x ToF • 2x Color • 2x UnitV2 AI cameras • servo rescue-kit dropper

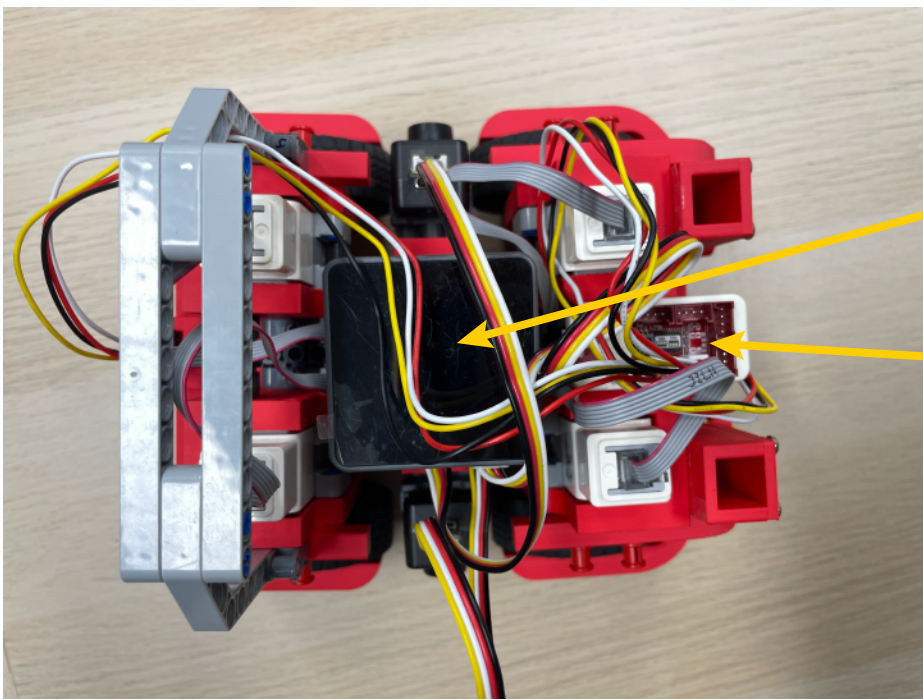
THE TEAM



- [USEONG KIM, HAYOUNG OH]**
- Hardware & mechanics
- [SOO LEE, HAYOUNG OH]**
- Software (mapping & motion control)
- [SOO LEE, USEONG KIM]**
- Vision & sensors (cameras, color, ToF)

Awards / results:
3rd place in Rescue Maze at the Korean competition

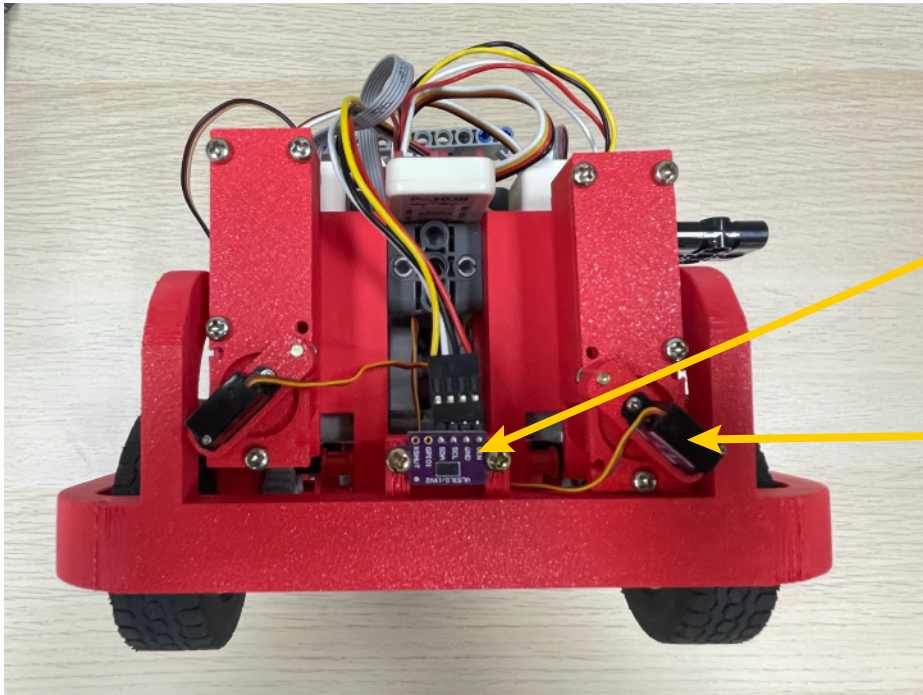
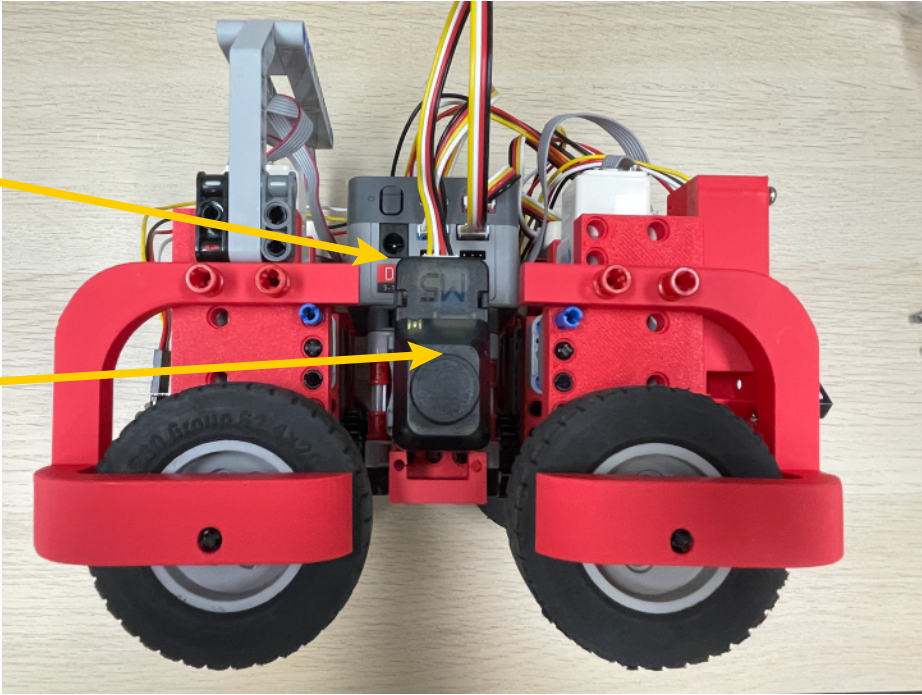
HARDWARE — ROBOT & SENSORS



- **Controller:** M5Stack Core S3 Lite (ESP32-S3, dual-core 240MHz, 9-axis IMU)
- **I2C hub:** PaHub (PCA9548A) — splits the six same-address (0x29) ToF & Color sensors

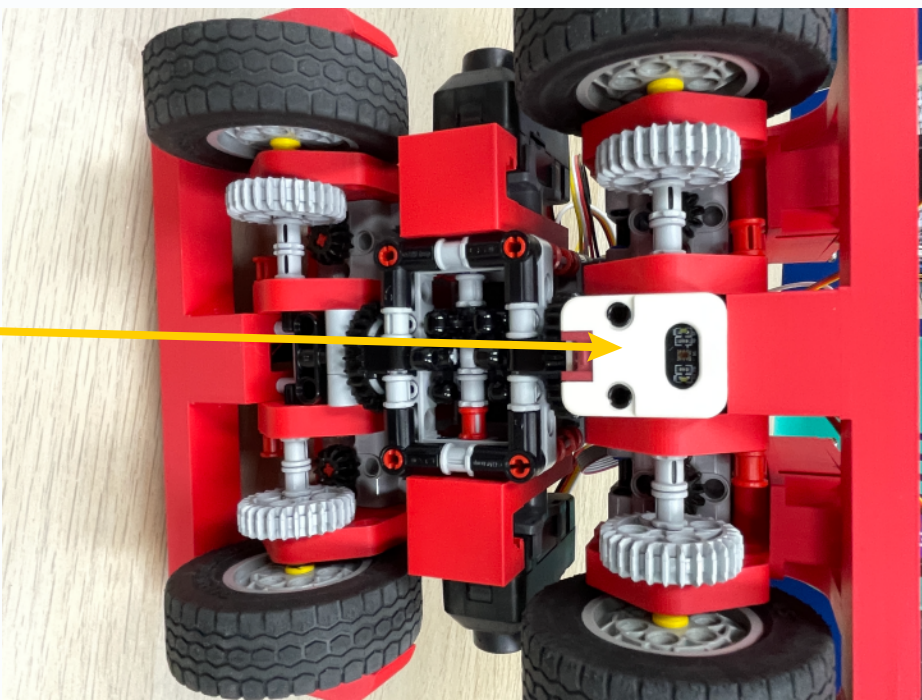
- **Drive:** BaseX (STM32, I2C) + 4x LEGO EV3 motors, 12T→28T gearing, 4WD

- **Vision:** 2x UnitV2 edge-AI cameras (left/right) — victim & target



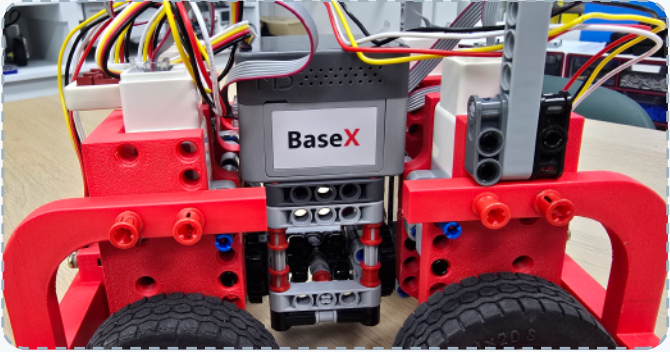
- **Distance:** 4x ToF VL53L0X (front/back/left/right) — wall sensing
- **Actuator:** 2x C017CLS micro-servo — rescue-kit dropper

- **Floor:** 2x Color TCS3472 — black-tile / checkpoint detection

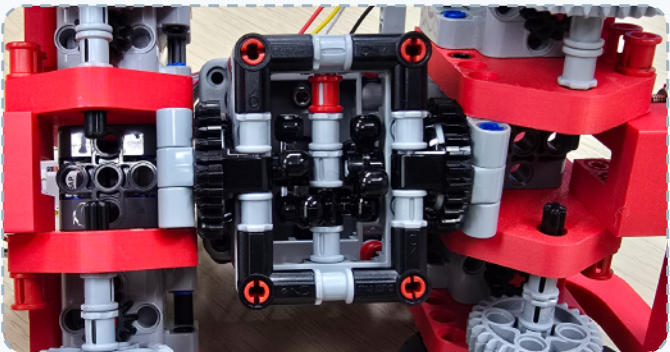


HARDWARE HIGHLIGHTS

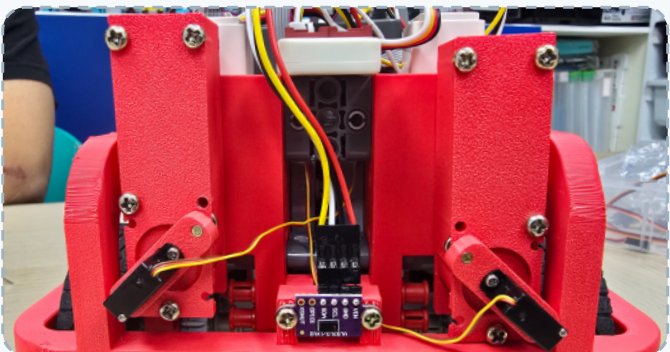
Innovative mechanisms (Rescue Maze):



CUSTOM DRIVE — BaseX + EV3
No ready-made MicroPython driver exists for the BaseX motor base, so we wrote our own from its I2C register map (basex.py: straight, drive, encoder odometry). Gyro-PD heading control is layered on top.



DIFFERENTIAL SUSPENSION
A central differential lets the left and right sides flex independently, keeping all four wheels on the ground over speed bumps and uneven tiles — reliable traction and odometry

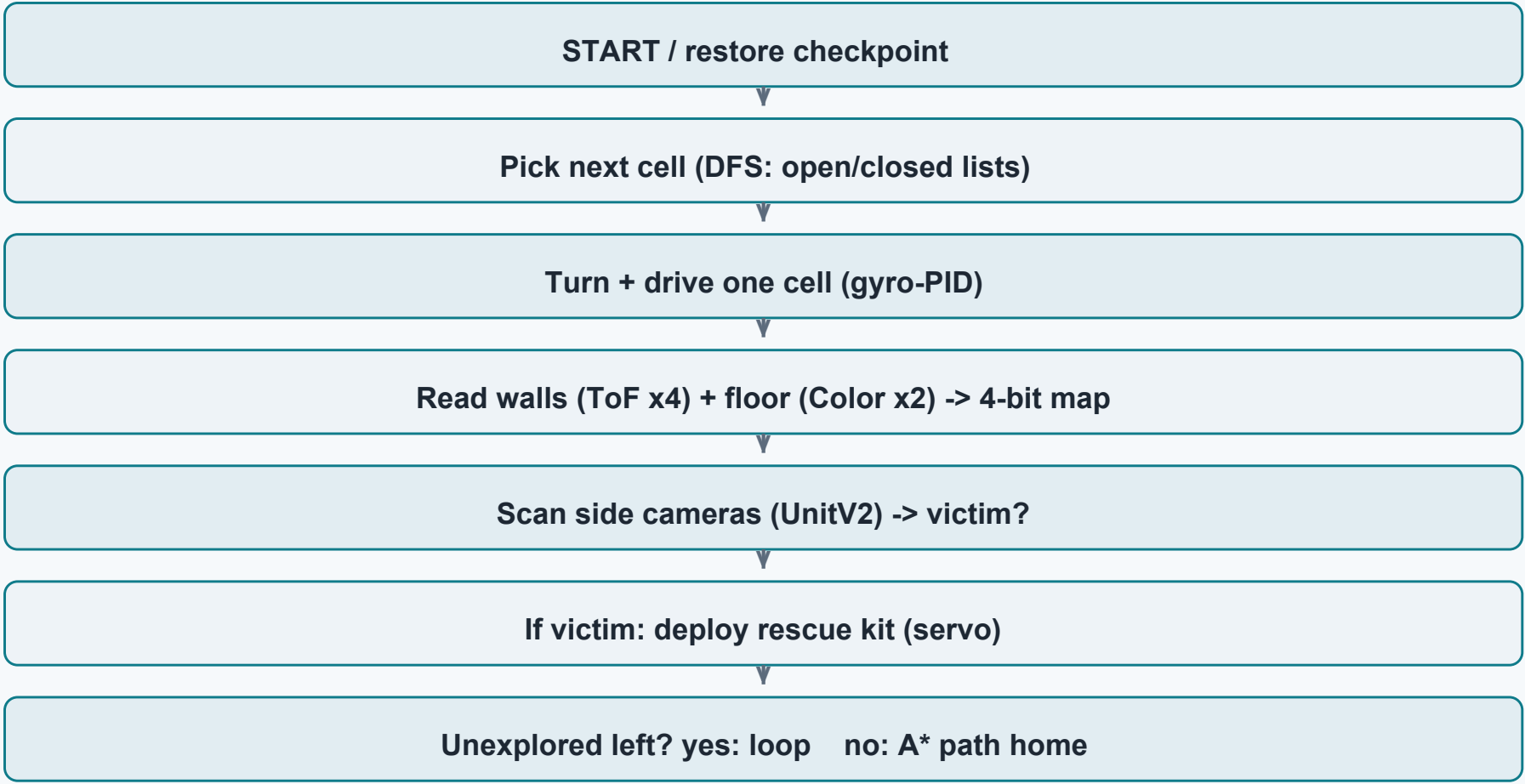


SERVO RESCUE-KIT DROPPER
Two micro-servos deploy rescue kits to the left / right wall when a victim is detected.

ROBOT CAPABILITIES

- Fully autonomous maze exploration with **complete-coverage mapping**
- Precise cell-by-cell navigation with **gyro-PD** straight driving & 90° turns
- Detects **letter victims (Φ/Ψ/Ω)** and **cognitive targets** (colour)
- **Rescue-kit deployment** via servo dropper
- Speed-bump traversal via differential suspension
- Black-tile avoidance, **checkpoint recovery** after lack-of-progress resets

SOFTWARE — MAIN LOOP



SOFTWARE — ALGORITHMS & TESTS

- **Map:** 2D grid; each cell is a **4-bit wall code** (N/S/W/E); grows from the start cell (auto-expanding)
- **Explore:** DFS with open/closed lists; **A*** (heapq, Manhattan h) for the path home
- **Motion:** our own BaseX **DriveBase** (4 motors); **gyro-PID** straight driving & 90° turns
- **IMU:** BMI270 gyro — integration + 1° deadzone + auto bias re-zero
- **Robust:** checkpoint save/restore after a reset; bidirectional drive avoids 180° turns
- **Runtime:** MicroPython, threaded I2C bus + uasyncio; 2x UnitV2 cameras for victim ID

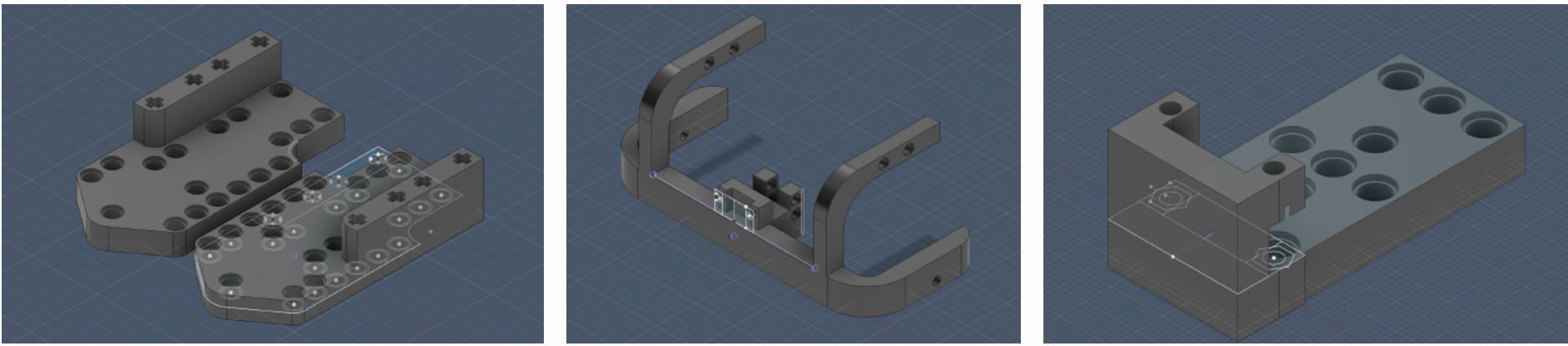
- How we test:**
- Desktop A* grid visualiser (astar/visualize.py) before hardware
 - On-maze tuning: cell distance, gyro 90° turns, ToF wall thresholds
 - [TEAM: add measured results — laps completed, victims found, reliability %]

SPECIFICATIONS

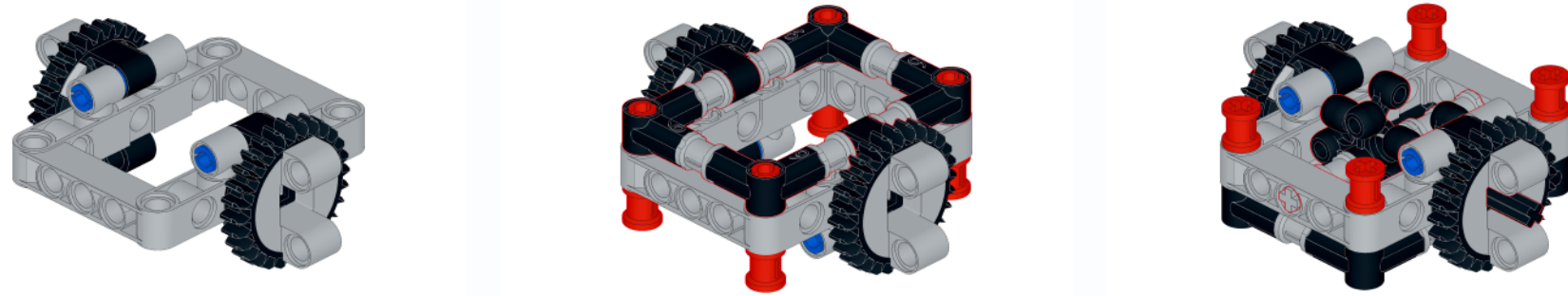
Controller	M5Stack Core S3 Lite (ESP32-S3)
Language	MicroPython + Python (sim)
Sensors	4x ToF, 2x Color, 2x UnitV2, 9-axis IMU
Construction	LEGO Technic + 3D-printed PLA (Fusion 360)
Development time	Early Apr 2026 - Jul 2026 (~3 months)
Material cost	[from BOM total — TEAM TO FILL]

DEVELOPMENT & BUILD PROCESS

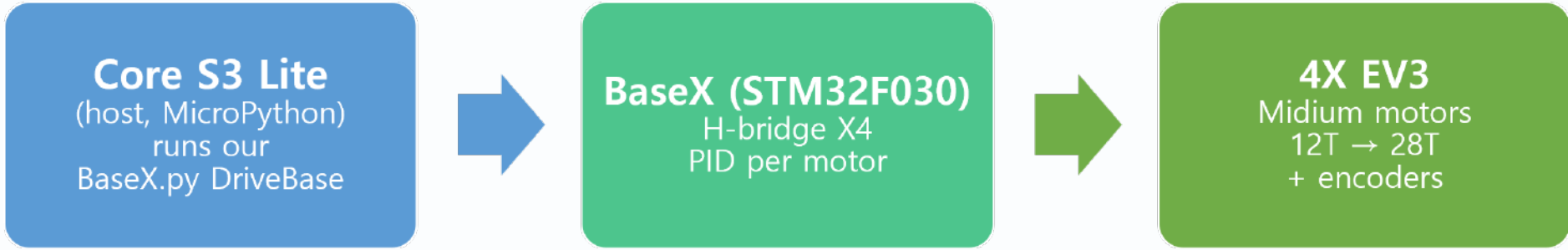
- Designed custom parts in **Fusion 360**, printed in **PLA** (sensor mounts, dropper, brackets)



- Built drivetrain on a **LEGO Technic** chassis with differential suspension



- Wrote our own **BaseX motor driver** from its I2C register map



- Verified the maze logic on desktop (grid visualiser) and a motors-disabled dry-run before hardware

